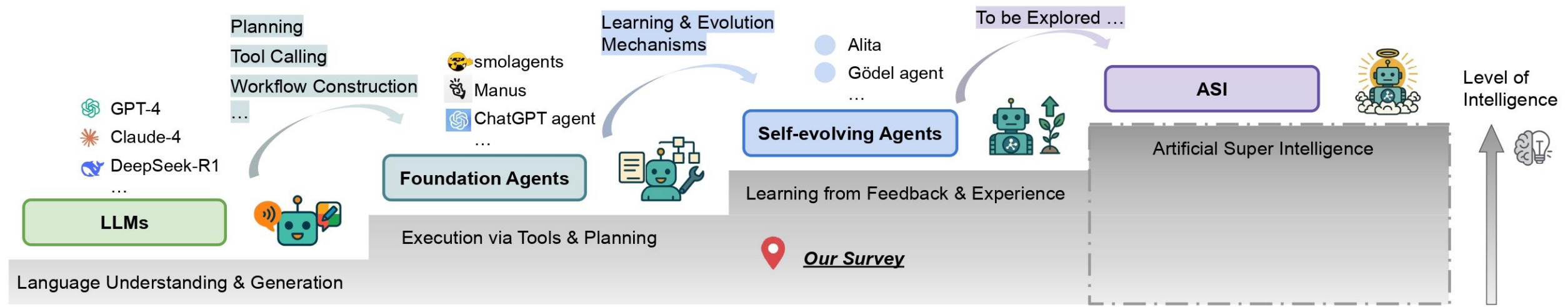
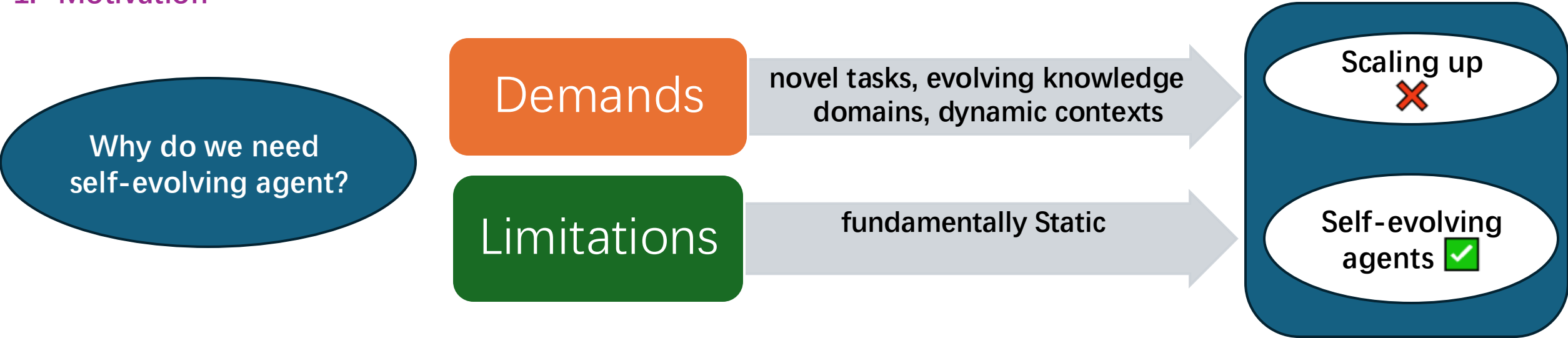


Continual Adaptation via Prompt Evolution

Wei Chen
12/05/2025

A Survey of Self-Evolving Agents: On Path to Artificial Super Intelligence

1. Motivation



2. Definitions and Foundations

Environment We first define the environment (including the user and the execution environment, e.g., Linux shell) of an agent system as a partially observable Markov Decision Process (POMDP), represented as a tuple $E = (\mathcal{G}, \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma)$, where:

- $\mathcal{G}$ is a set of potential goals. Each $g \in \mathcal{G}$ is a task objective that the agent needs to achieve, e.g., a user query.
- $\mathcal{S}$ is a set of states. Each $s \in \mathcal{S}$ represents the internal state of the environment.
- $\mathcal{A}$ is a set of actions. Each action $a \in \mathcal{A}$ can be a combination of textual reasoning, retrieval of external knowledge, and tool calls.
- T is the state transition probability function which takes a state-action pair (s, a) and outputs the probability distribution $T(s'|s, a)$ of the next state.
- $R : \mathcal{S} \times \mathcal{A} \times \mathcal{G} \rightarrow \mathcal{R}$ is the feedback/reward function, conditioned on the specific goal $g \in \mathcal{G}$. The feedback $r = R(s, a, g)$ typically takes the form of a scalar score or textual feedback.
- Ω is a set of observations accessible to the agent.
- O is the observation probability function which takes a state-action pair (s, a) and outputs the probability distribution $O(o'|s, a)$ of the next observation for the agent.
- γ is the discount factor.

Agent system We define a (multi-)agent system as $\Pi = (\Gamma, \{\psi_i\}, \{C_i\}, \{\mathcal{W}_i\})$. The architecture Γ determines the control flow of the agent system or collaborative structures between multiple agents. It is typically represented as a sequence of nodes $(N_1, N_2, \dots)$ organized by graph or code structures. Each node N_i consists of the following components:

- ψ_i : the underlying LLM/MLLM.
- C_i : the context information, e.g., prompt P_i and memory M_i .
- $\mathcal{W}_i$: the set of available tools/APIs.

At each node, the agent policy is a function $\pi_{\theta_i}(\cdot|o)$ that takes an observation and outputs the probability distribution of the next action, where $\theta_i = (\psi_i, C_i)$. The actual action space here is the union of the natural language space and the tool space $\mathcal{W}_i$.

For a given task $\mathcal{T} = (E, g)$, represented by an environment E and a corresponding goal $g \in \mathcal{G}$, the agent system follows the topology Γ to generate a trajectory $\tau = (o_0, a_0, o_1, a_1, \dots)$, and receives a feedback r either from the external environment or from internal signals (e.g., self-confidence or feedback from an evaluator).

Self-evolving strategy A self-evolving strategy is a transformation f that maps the current agent system to a new state, conditioned on the generated trajectory τ and the external/internal feedback r :

$$f(\Pi, \tau, r) = \Pi' = (\Gamma', \{\psi'_i\}, \{C'_i\}, \{\mathcal{W}'_i\}) \quad (1)$$

Objective of self-evolving agents Let U be a utility function that measures the performance of an agent system Π on a given task $\mathcal{T}$ by assigning a scalar score $U(\Pi, \mathcal{T}) \in \mathbb{R}$. The utility may be derived from the task-specific feedback r , such as a reward signal or textual evaluation, possibly combined with other performance indicators (e.g., completion time, accuracy, or robustness). Given a sequence of tasks $(\mathcal{T}_0, \mathcal{T}_1, \dots, \mathcal{T}_n)$ and an initial agent system Π_0 , a self-evolving strategy f recurrently generates an evolving sequence of agent systems $(\Pi_1, \Pi_2, \dots, \Pi_n)$ via

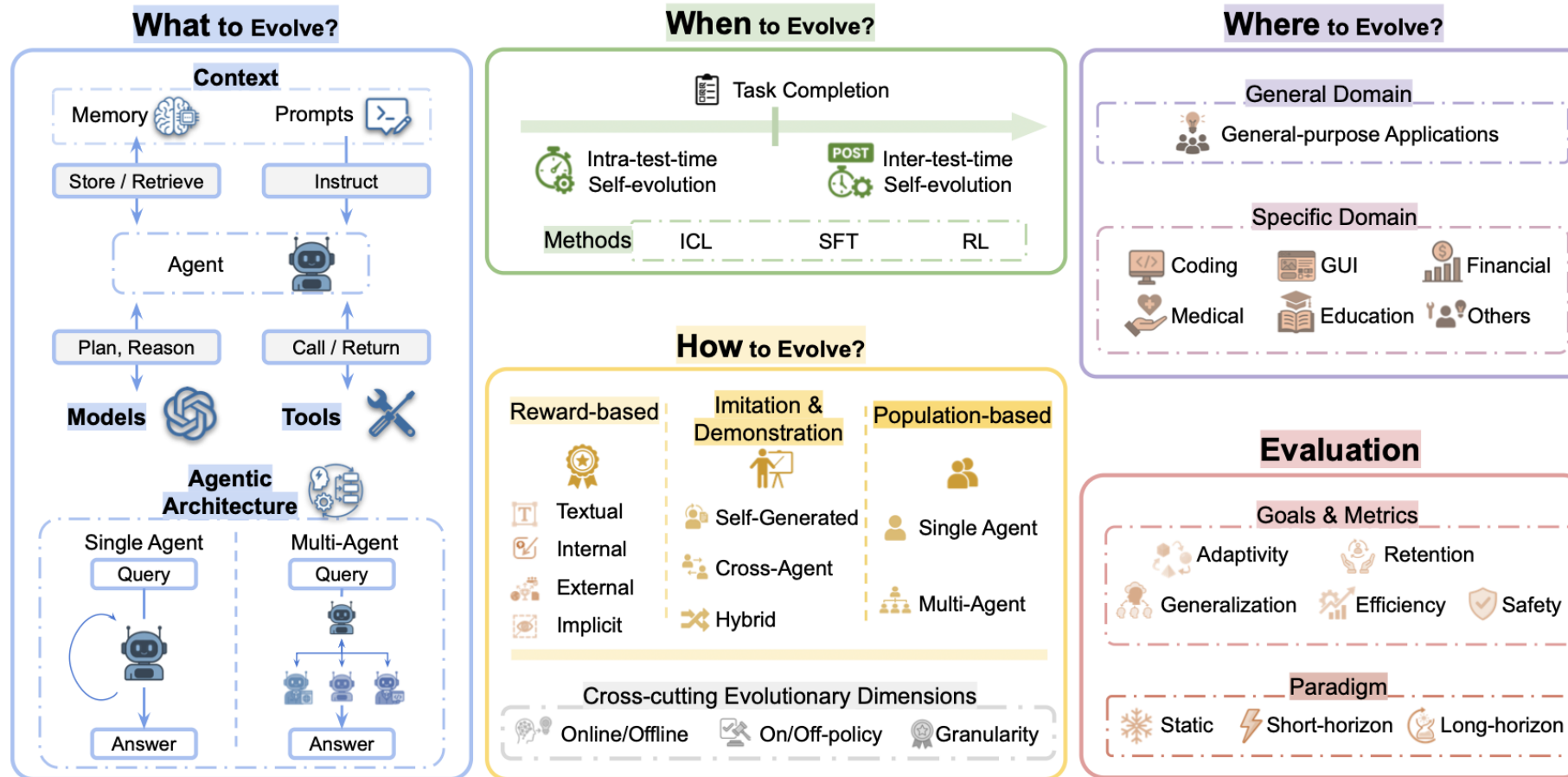
$$\Pi_{j+1} = f(\Pi_j, \tau_j, r_j), \quad (2)$$

where τ_j and r_j are the trajectory and feedback on task $\mathcal{T}_j$.

The overarching objective in designing a self-evolving agent is to construct a strategy f such that the cumulative utility over tasks is maximized:

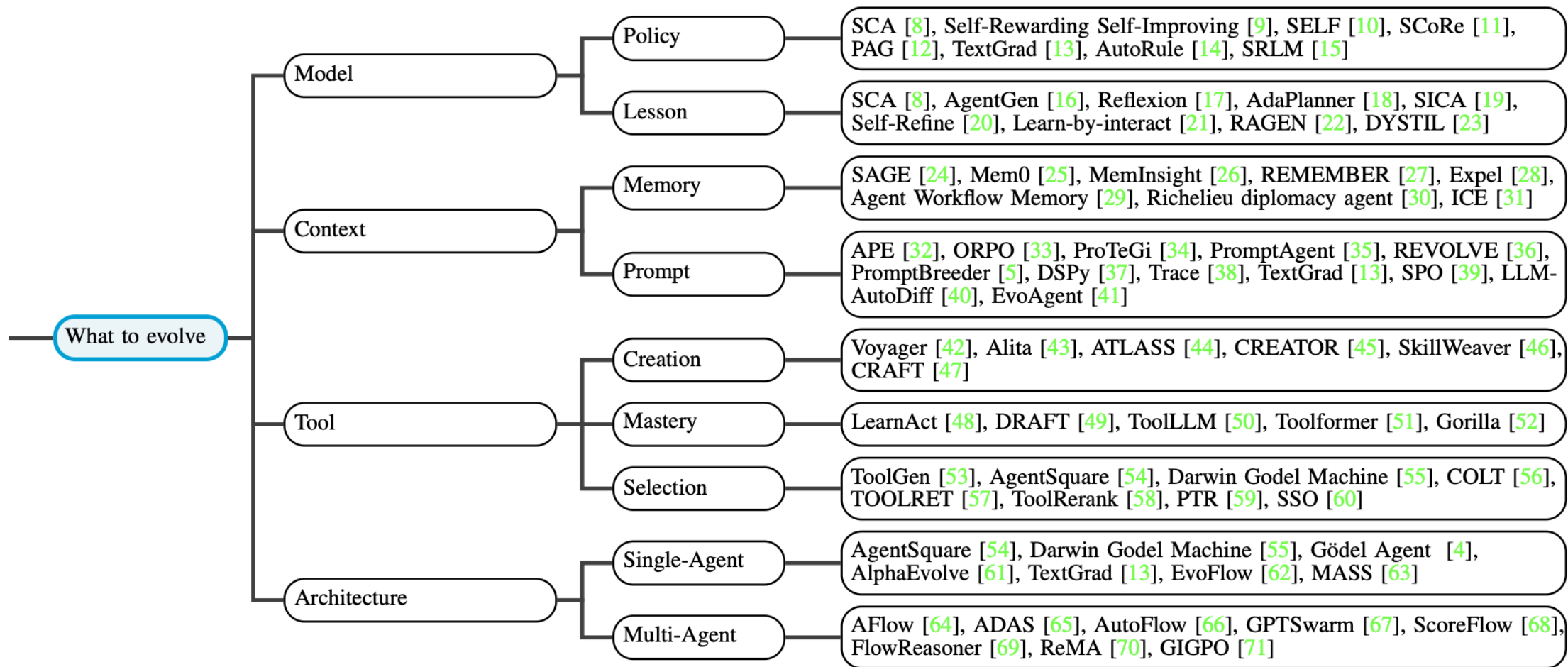
$$\max_f \sum_{j=0}^n U(\Pi_j, \mathcal{T}_j) \quad (3)$$

3. Four key dimensions and evaluation:



What aspects of an agent should evolve?

$$\Pi = (\Gamma, \{\psi_i\}, \{C_i\}, \{\mathcal{W}_i\})$$



When should adaptation occur?

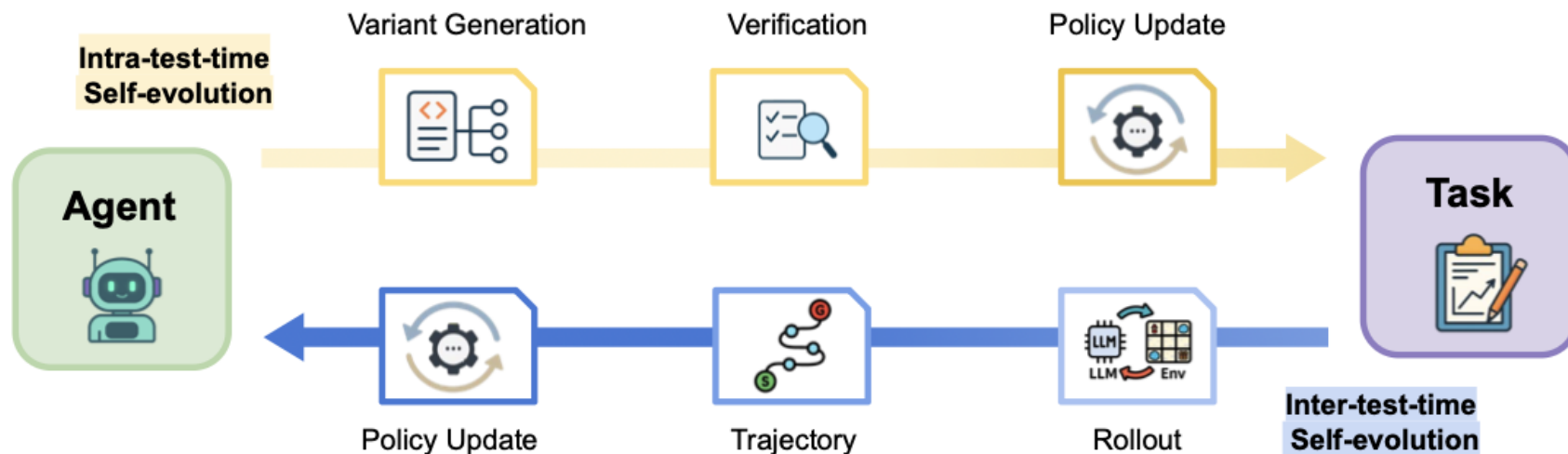


Figure 5: An overview of when to evolve. The top pathway illustrates intra-test-time self-evolution, where adaptation (e.g., variant generation, verification, and policy update) occurs within task execution. The bottom pathway depicts inter-test-time self-evolution, where learning happens retrospectively through rollout, trajectory analysis, and policy updates.

How should that evolution be implemented in practice?

$$f(\Pi, \tau, r) = \Pi' \quad \text{where} \quad \Pi = (\Gamma, \{\psi_i\}, \{C_i\}, \{\mathcal{W}_i\}), \quad \Pi' = (\Gamma', \{\psi'_i\}, \{C'_i\}, \{\mathcal{W}'_i\})$$

U is a utility function that assigns a scalar score $U(\Pi, \mathcal{T}) \in \mathbb{R}$ to an agent system Π on task $\mathcal{T}$.

The overarching objective in designing a self-evolving agent is to construct a strategy f such that the cumulative utility over tasks is maximized:

$$\max_f \sum_{j=0}^n U(\Pi_j, \mathcal{T}_j) \tag{3}$$

Overview:

1. Reward-based evolution

2. Imitation & Demonstration

3. Population-based/Evolutionary Methods

Feedback type: Language, Numerical or Both

Feedback source: Internal, External or Both

Popular learning **methods:** ICL SFT RL

Updated timing: test-time, Pre-test time or Both

Table 4: Comparison of self-evolution method families along key dimensions.

Dimension	Reward-based	Imitation/Demonstration	Population-based
Feedback Type	Scalar reward, natural language, confidence, external signals	Demonstration trajectories, exemplars, rationales	Fitness scores, task success, competitive signals
Data Source	Self-generated, environment, external rules	Self-generated or other agents, humans	Population generations, multi-agent systems
Reward Granularity	Outcome/process/hybrid (flexible)	Usually outcome/process (via demo steps)	Often outcome-level, sometimes process via competition
Online/Offline	Both (reward learning, RL, DPO, SFT)	Typically offline, sometimes online demo mining	Online evolution or batch population updates
On/Off-policy	Both (DPO, Reflexion, GRPO)	Primarily off-policy, but online variants can be on-policy	Off-policy (population); self-play is on-policy
Sample Efficiency	Moderate (depends on reward sparsity)	High (if demo quality is high)	Usually low (needs many trials)
Stability	Sensitive to reward design	Sensitive to demo quality/diversity	Sensitive to population size/diversity
Scalability	Good with automation	Limited by demo collection	High but resource-intensive

Where to evolve?

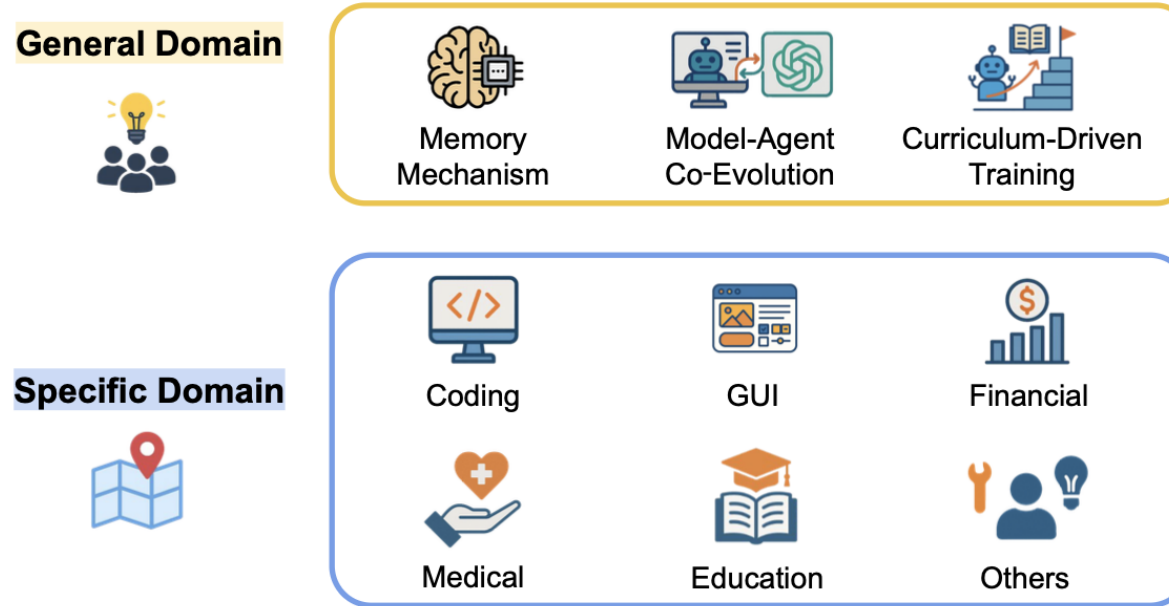


Figure 8: Categorization of where to evolve into two major types: General Domain Evolution, which focuses on broad capability enhancement across diverse tasks (e.g., memory mechanisms, co-evolution, curriculum training), and Specific Domain Evolution, which targets domain-specific expertise in areas such as coding, GUI, finance, medical, education, and others.

4. Evaluation of Self-evolving Agents

their evaluation:

- *not only* immediate task success

but also:

- **adaptation** over time
- **knowledge accumulation and retention**
- **long-term generalization**
- **transfer learned skills across sequential or novel tasks,**
- **mitigating catastrophic forgetting**

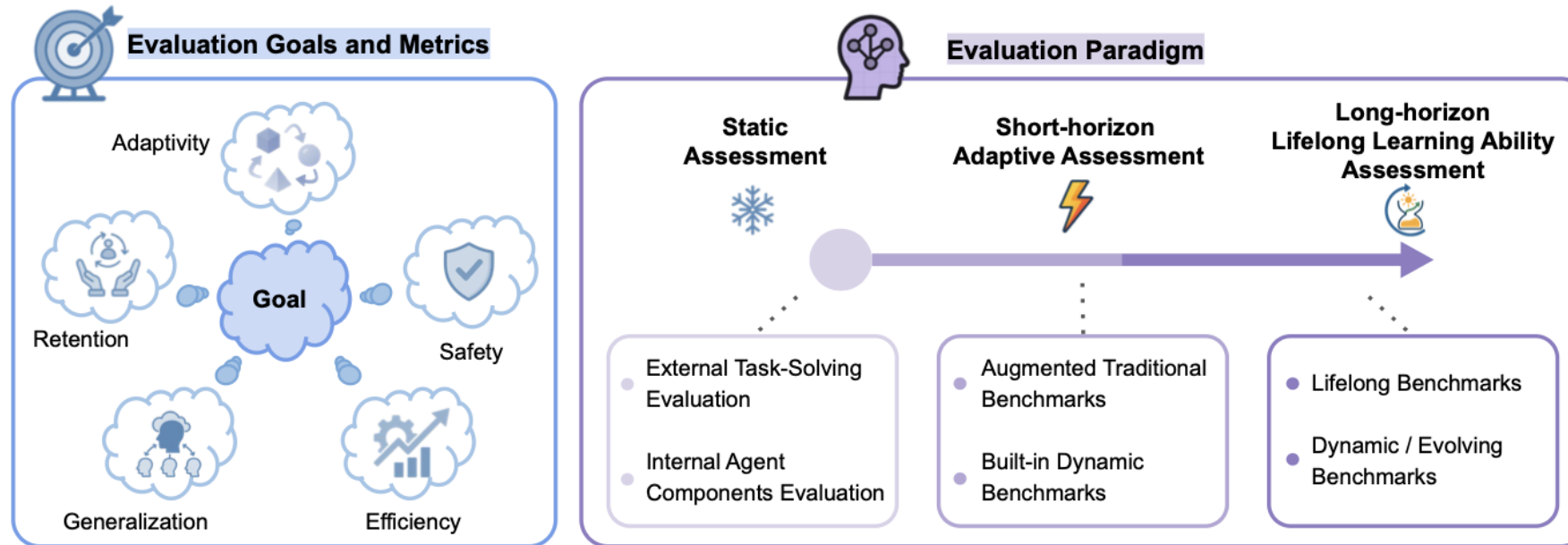


Figure 9: Overview of evaluation angles for self-evolving agents, encompassing core Evaluation Goals and Metrics—such as adaptivity, retention, generalization, safety, and efficiency—and a continuum of Evaluation Paradigms spanning from static assessment to short-term adaptability and long-horizon lifelong learning evaluation.

PromptBreeder: Self-Referential Self-Improvement via Prompt Evolution

Why PromptBreeder? What problem does PromptBreeder solve?

- Prompt Engineering Is **Fragile & Manual**

Chain-of-Thoughts: hand-crafted, highly depends on human experience,
new tasks $\longrightarrow$ redesign

- Existing Automatic Methods **Plateau Quickly**

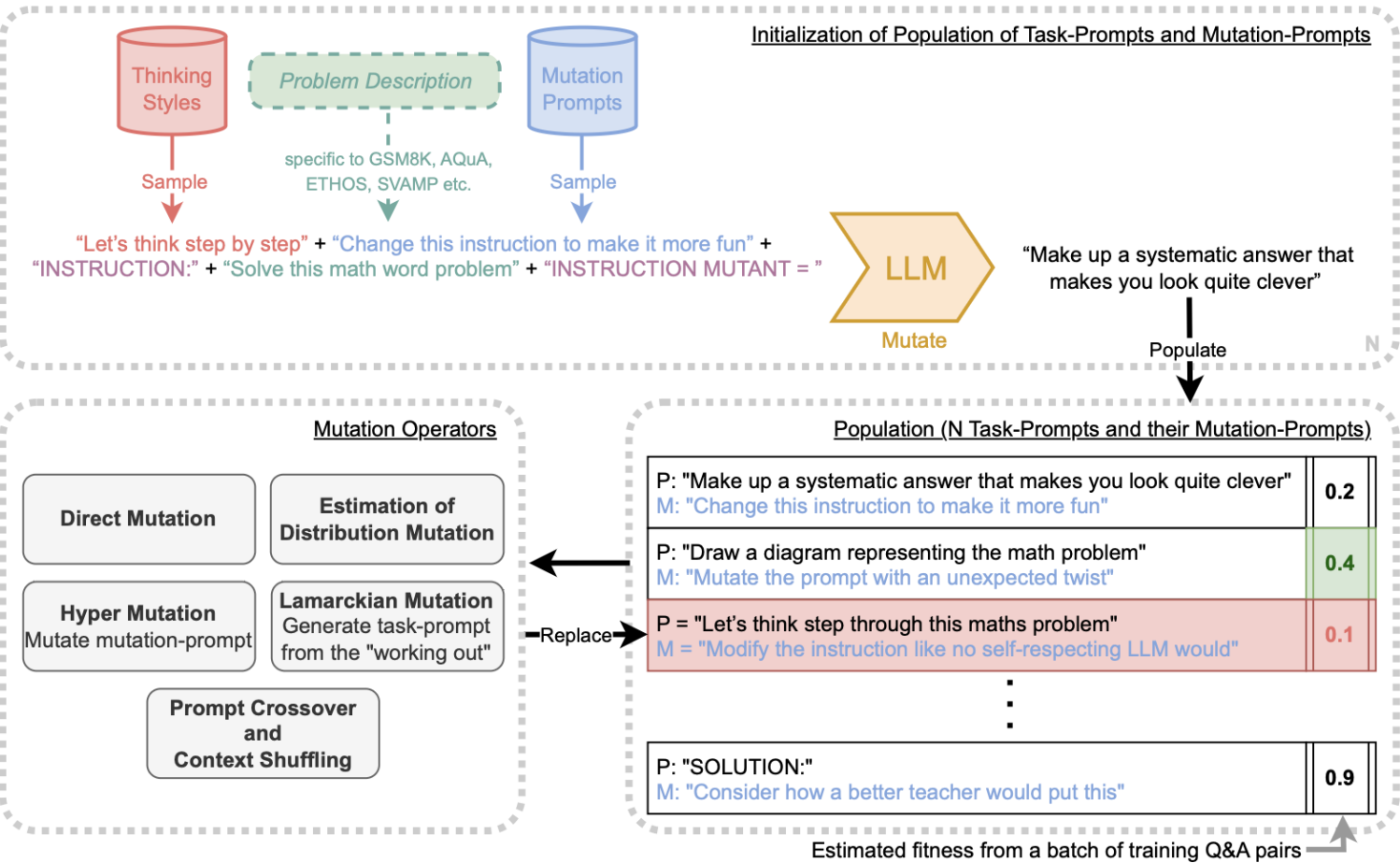
Auto-CoT, APE, or OPRO that try to generate or optimize prompts automatically
very local neighborhood $\longrightarrow$ diminishing

- Prompt = Behavior Control for LLM

Prompt determines the model's reasoning process;
therefore, a prompt is fundamentally an **optimizable *control policy***.

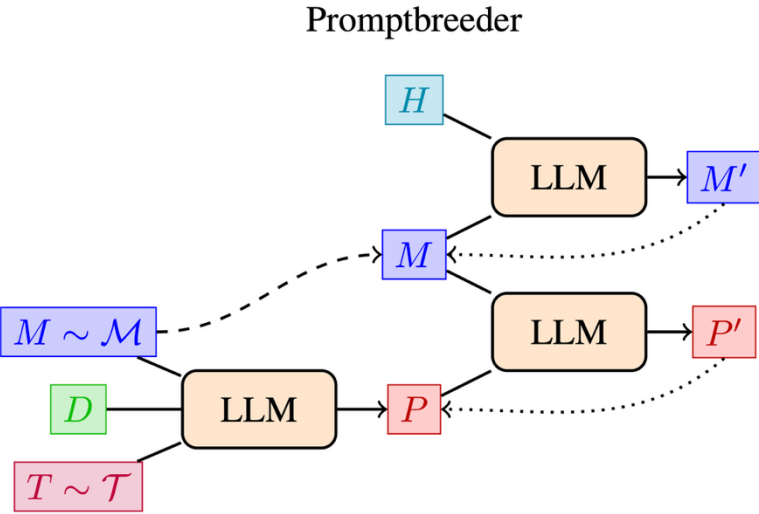
build a system that **continuously evolves** better CoT-style prompts by itself

Key Idea: Self-Referential, Self-Improvement Mechanism

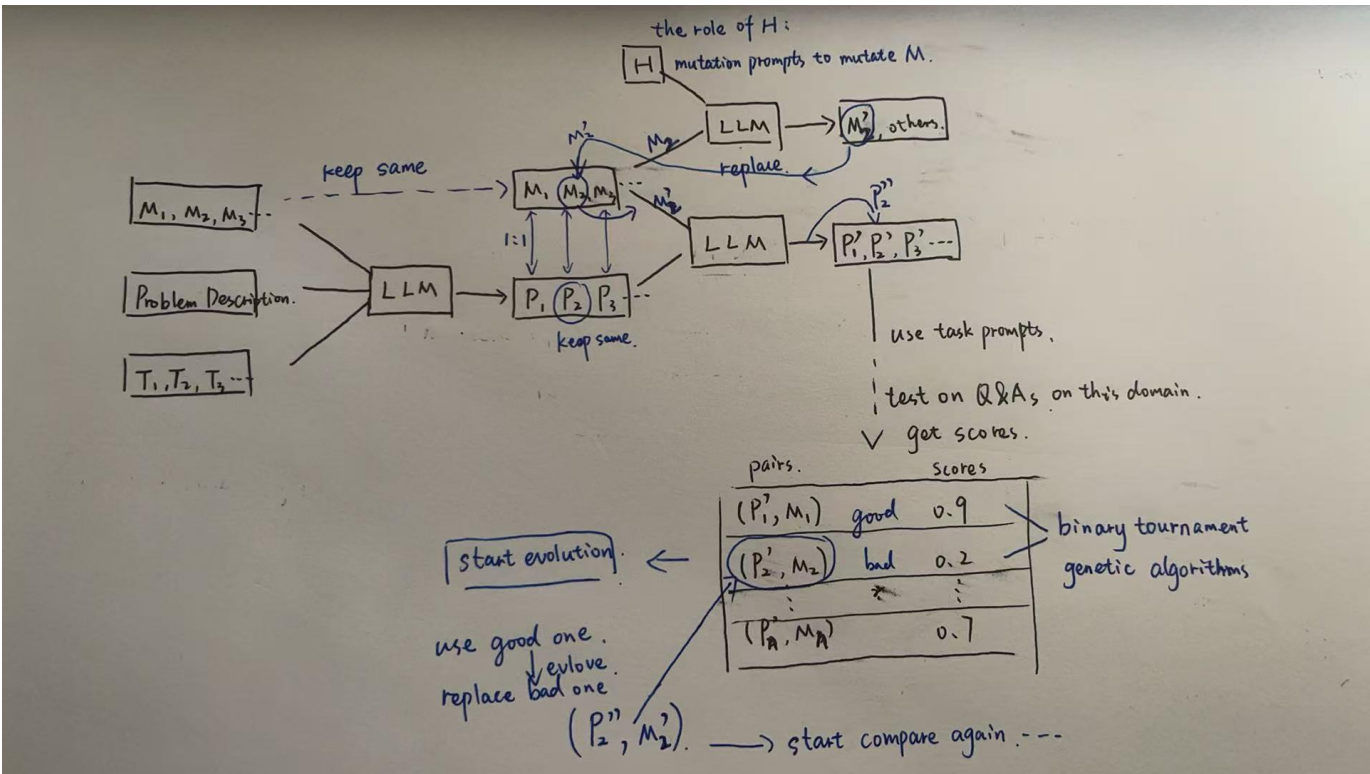


Essence: A Two-Level Self-Evolving System

evolve-the-prompt + evolve-the-evolver



Key Idea: Self-Referential, Self-Improvement Mechanism



Question:

Which step ensures diversity?

What is the difference if we do test after only 1st LLM?

Some sentences from Author:

LLM itself to assess and promote the **diversity** or to use it to determine the **fitness** of a whole “thought process”

Initialization:

Problem description

+
a set of thinking styles

+
A set of initial Mutation-Prompts

Mutation of Task-Prompts PromptBreeder evolves task-prompts using mutation-prompts M :

$$P' = \text{LLM}(M + P)$$

where “+” denotes string concatenation.

Self-Referential Mutation (Meta-Level) The mutation-prompts themselves are evolved via hyper-mutation prompts H :

$$M' = \text{LLM}(H + M)$$

Automatic Instruction Evolving for Large Language Models

1. Problem Statement

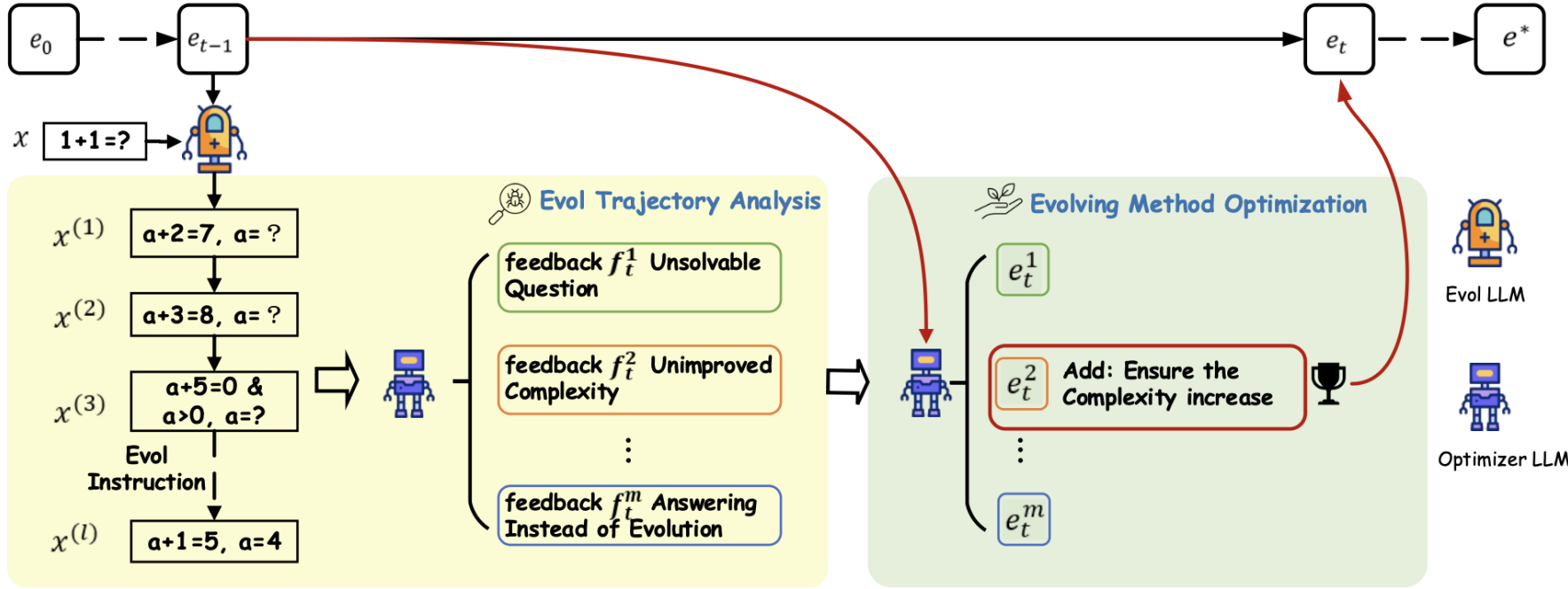
Traditional Evol-Instruct improves instruction datasets by using a human-designed evolving method e that transforms the original set X into an evolved set X_e :

$$X \xrightarrow{e} X_e.$$

Auto Evol-Instruct aims to *automatically* discover the evolving method that yields the best downstream performance:

$$e^* = \arg \max_e Q(X_e).$$

2. Method: Evol LLM + Optimize LLM



1. Multi-step Evolution At optimization step t , the evolving method e_{t-1} induces an l -step evolutionary trajectory over a batch X_t :

$$X_t \xrightarrow{e_{t-1}} X_t^{(1)} \xrightarrow{e_{t-1}} X_t^{(2)} \xrightarrow{e_{t-1}} \dots \xrightarrow{e_{t-1}} X_t^{(l)}.$$

Thus the trajectory is:

$$S_t = \{X_t, X_t^{(1)}, \dots, X_t^{(l)}\}.$$

2. Trajectory Analysis. The optimizer LLM analyzes S_t and generates m feedback signals:

$$f_t = \{f_t^1, \dots, f_t^m\}, \quad f_t^i = \text{Analyze}(S_t)_i.$$

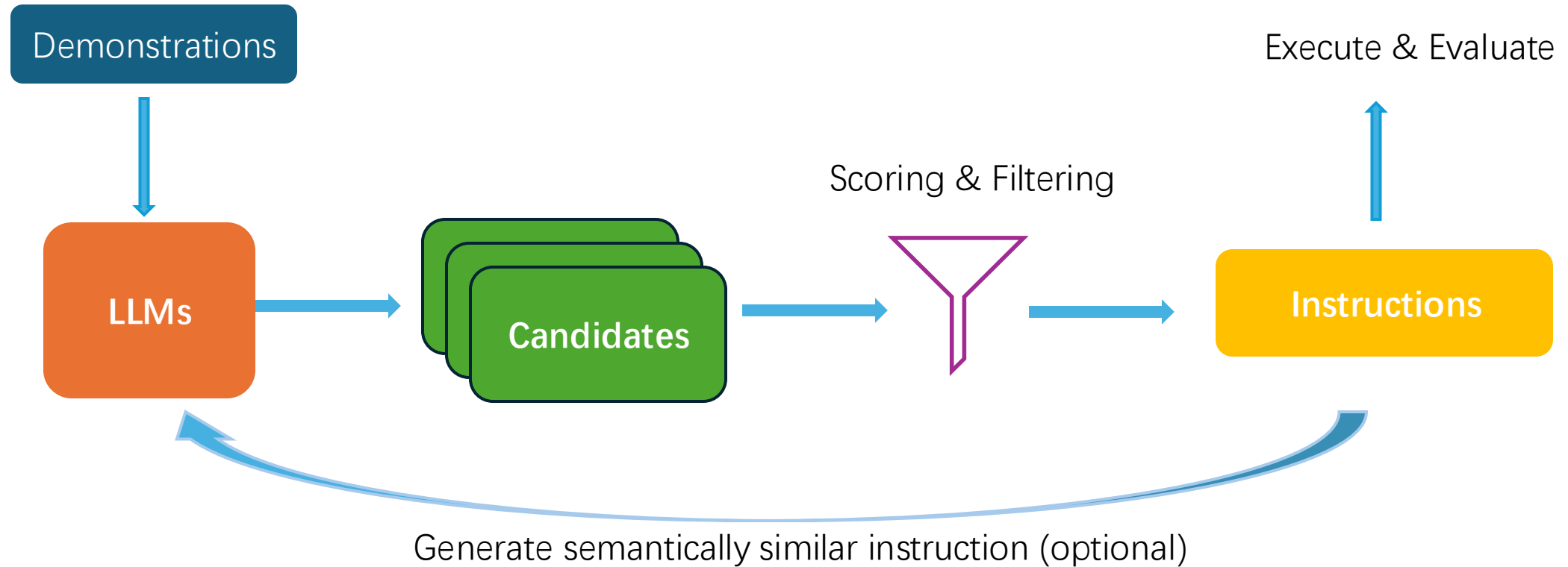
$$e_t^i = \text{Refine}(e_{t-1}, f_t^i).$$

$$\lambda(e_t^i) = \frac{1}{|D|} \sum_{r \in R_t(e_t^i)} F(r).$$

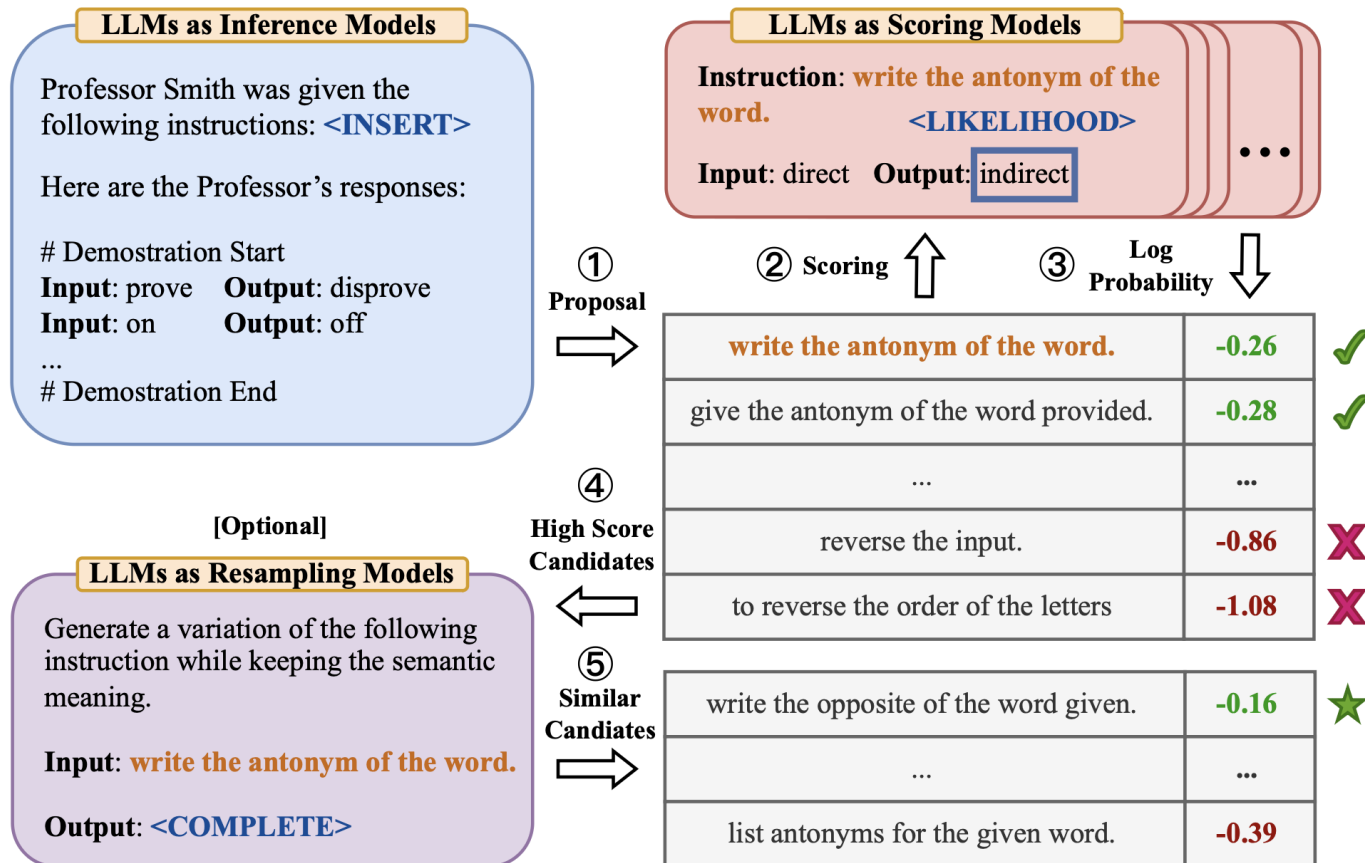
$$e_t = \arg \min_i \lambda(e_t^i).$$

Automatic Prompt Engineer (APE)

1. Workflow



✓ Keep the high score candidates ✗ Discard the low score candidates ★ Final selected prompt with highest score



Input	Output
sane	insane
direct	indirect
powerful	powerless
⋮	⋮
subtractive	additive
inexact	exact

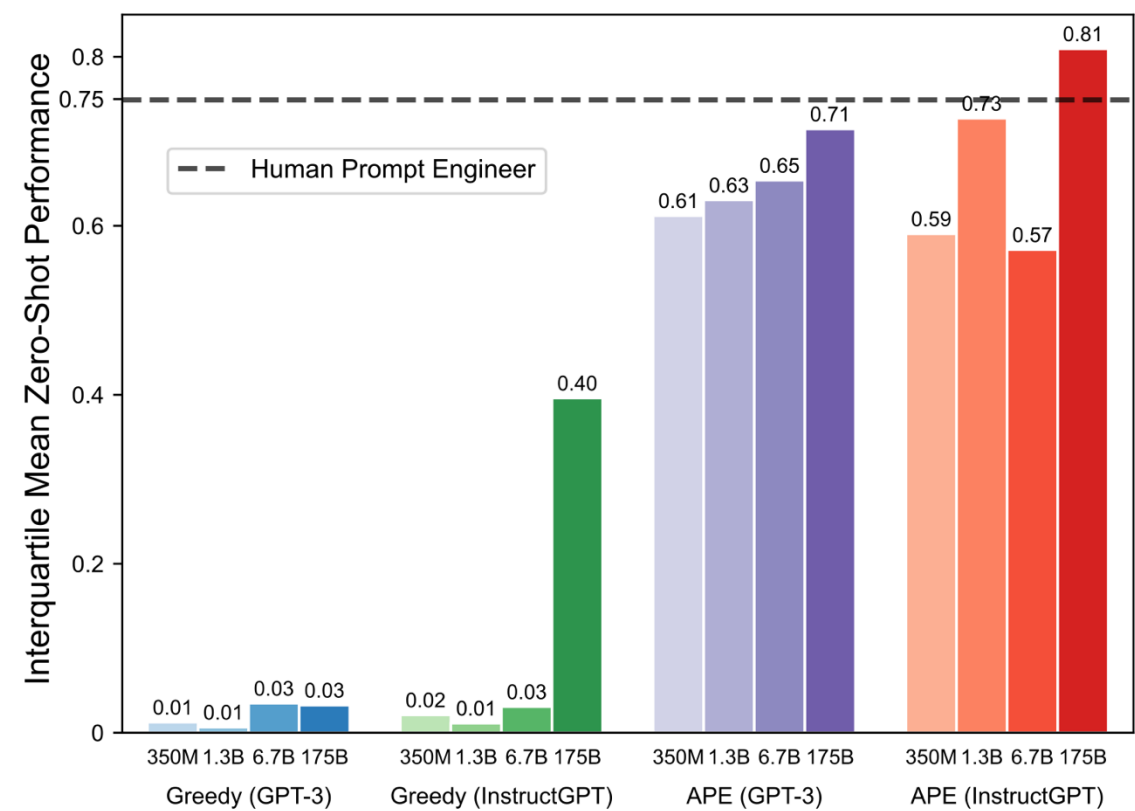
Zero-shot Evaluation

Instruction: write the opposite of the word given

Input: direct Output: <COMPLETE>

APE uses LLMs as: Inference Models, Scoring Models, Resampling Models

2. Some results:



(b) Interquartile mean across 24 tasks

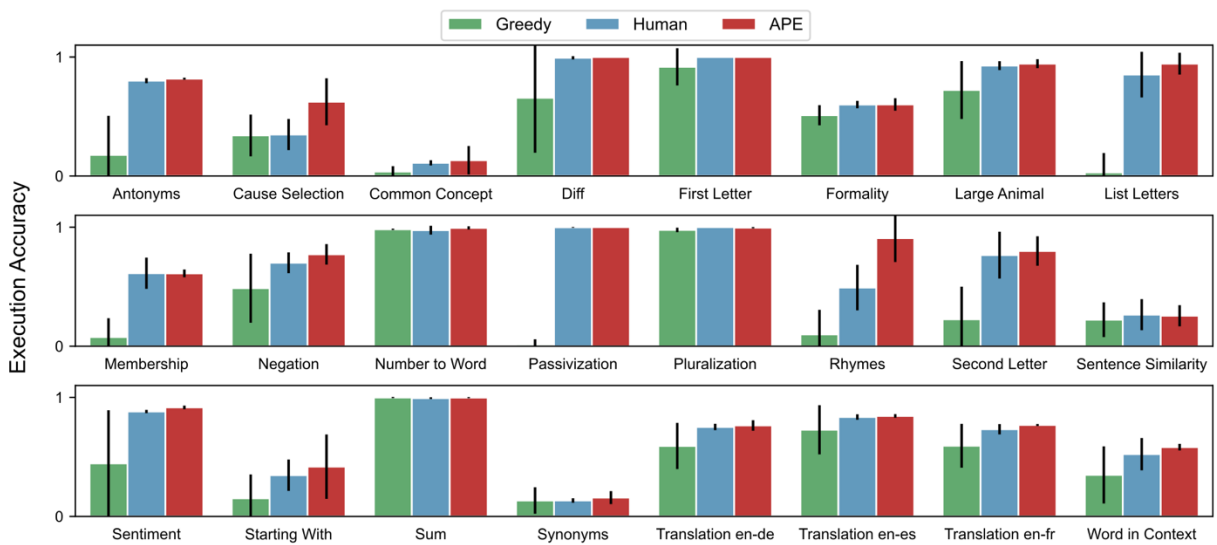
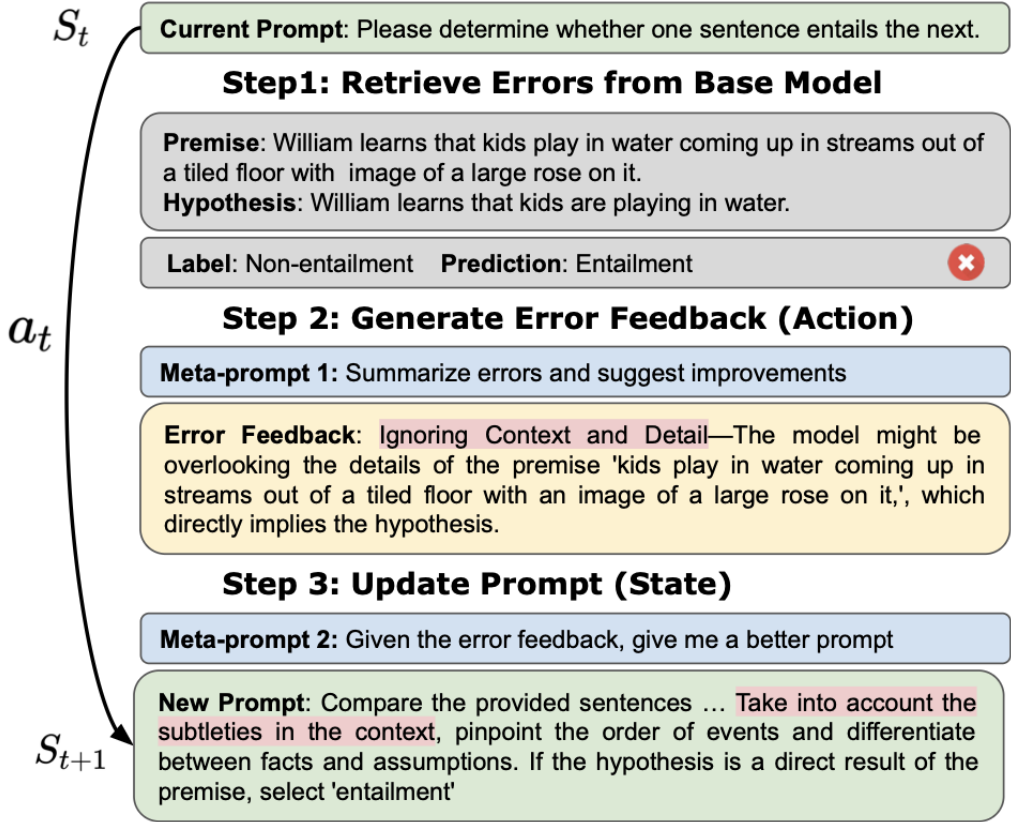


Figure 4: Zero-shot test accuracy on 24 Instruction Induction tasks. APE achieves human-level or better performance on all 24 out of 24 tasks.

PromptAgent: Strategy Planning with Language Models Enables Expert-Level Prompt Optimization

1. State Transition Example

(b) State Transition



Problem Formulation

Given an initial prompt P_0 and a labeled set

$$(Q, A) = \{(q_i, a_i)\}_{i=1}^N,$$

the base LLM predicts

$$p_B(a_i | q_i, P).$$

The optimization goal is

$$P^* = \arg \max_{P \in S} \sum_{i=1}^N R(p_B(a_i | q_i, P)).$$

State Transition

State:

$$s_t = P_t.$$

Action (error feedback):

$$a_t \sim p_O(a | s_t, m_1).$$

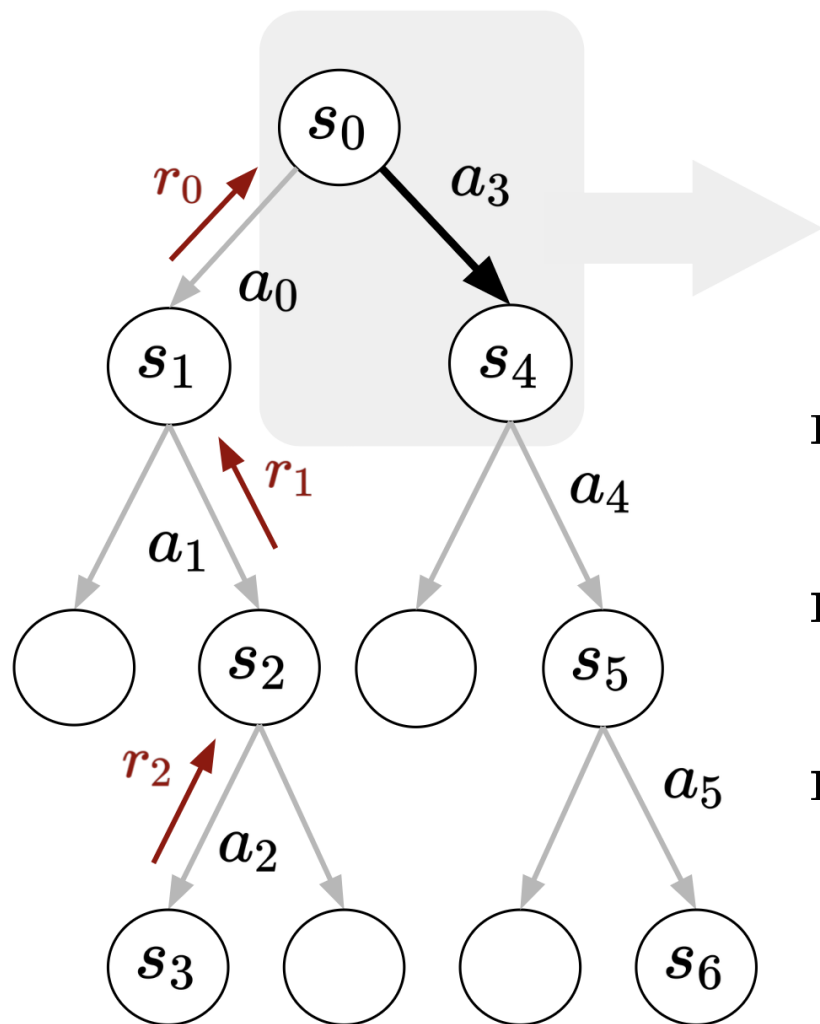
State update (new prompt):

$$s_{t+1} \sim p_O(s | s_t, a_t, m_2).$$

Reward:

$$r_t = r(s_t, a_t).$$

2. MCTS (Monte Carlo Tree Search) planning for expert-level prompting



Selection

$$a_t^* = \arg \max_{a \in A(s_t)} \left[Q(s_t, a) + c \sqrt{\frac{\ln N(s_t)}{N(\text{ch}(s_t, a))}} \right] \quad (\text{select best action})$$

Expansion / Transition

$$a_t \sim p_O(a \mid s_t, m_1), \quad s_{t+1} \sim p_O(s \mid s_t, a_t, m_2) \quad (\text{generate new prompt state})$$

Reward

$$r_t = r(s_t, a_t) \quad (\text{evaluate prompt})$$

Backpropagation

$$Q(s_t, a_t) = \frac{1}{M} \sum_{j=1}^M \sum_{(s,a) \in \tau_j} r(s, a) \quad (\text{propagate reward upward})$$

Algorithm 1 PromptAgent-MCTS($s_0, p_\theta, r_\theta, p_\phi, d, L, \tau, c$)

Inputs:

Initial prompt (state) s_0 , state transition function p_θ , reward function r_θ , action generation function p_ϕ , number of generated actions d , depth limit L , iteration number τ , exploration weight c (Equation 1)

Initialize:

State to action mapping $A : \mathcal{S} \mapsto \mathcal{A}$, children mapping $\text{ch} : \mathcal{S} \times \mathcal{A} \mapsto \mathcal{S}$, rewards $r : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$, State-action value function $Q : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$, visit-time counter $\mathcal{N} : \mathcal{S} \mapsto \mathbb{N}$

for $n \leftarrow 0, \dots, \tau - 1$ **do**

for $t \leftarrow 0, \dots, L - 1$ **do**

if $A(s_t)$ is not empty **then**

▷ selection

$a_t \leftarrow \arg \max_{a \in A(s_t)} \left(Q(s_t, a) + c \cdot \sqrt{\frac{\ln \mathcal{N}(s_t)}{\mathcal{N}(\text{ch}(s_t, a))}} \right)$
 $s_{t+1} \leftarrow \text{ch}(s_t, a_t), r_t \leftarrow r(s_t, a_t), \mathcal{N}(s_t) \leftarrow \mathcal{N}(s_t) + 1$

else

▷ expansion and simulation

for $i \leftarrow 1, \dots, d$ **do**

 Sample $a_t^i \sim p_\phi(a|s_t), s_{t+1}^i \sim p_\theta(s|s_t, a_t^i)$, and $r_t^i \leftarrow r_\theta(s_t, a_t^i)$

 Update $A(s_t) \leftarrow \{a_t^i\}_{i=1}^d, \text{ch}(s_t, a_t^i) \leftarrow s_{t+1}^i$, and $r(s_t, a_t^i) \leftarrow r_t^i$

end for

$a_t \leftarrow \arg \max_{a_t^i \in A(s_t)} r_t^i(s_t, a_t^i)$

$s_{t+1} \leftarrow \text{ch}(s_t, a_t), r_t \leftarrow r(s_t, a_t), \mathcal{N}(s_t) \leftarrow \mathcal{N}(s_t) + 1$

end if

if s_{t+1} is an early-stopping state **then break**

end for

$T \leftarrow$ the actual number of steps

for $t \leftarrow T - 1, \dots, 0$ **do**

▷ back-propagation

 Update $Q(s_t, a_t)$ with $\{r_t, r_{t+1}, \dots, r_L\}$ based on Equation 2

end for

end for
